

Task 1: Training with PyTorch

Description: Implement one training iteration using PyTorch's built-in automatic differentiation. The task demonstrates how to:

- Create a simple neural network with one hidden layer (4→8→3 architecture)
- Perform forward propagation through the network
- Calculate cross-entropy loss
- Compute gradients automatically using backpropagation
- Update weights using SGD optimizer

Key Components:

- **Initial Weights:** Random initialization using PyTorch's default method
- **Gradients:** Automatically computed via `loss.backward()`
- **Weight Updates:** Applied using `optimizer.step()` with learning rate 0.01

Task 2: Training with NumPy

Description: Implement the same training iteration from scratch using only NumPy. This task reveals the mathematical operations happening under the hood:

- Manual forward propagation
- Manual gradient computation using calculus chain rule
- Manual weight updates using gradient descent

Key Components:

- **Initial Weights:** Random initialization using NumPy
- **Gradients:** Manually calculated using:
 - Softmax derivative for output layer
 - Tanh derivative for hidden layer
 - Chain rule to backpropagate through layers
- **Weight Updates:** Manual application of gradient descent formula: $W = W - \eta r * dW$

Both implementations train on the same 5-sample dataset with 4 features, classifying into 3 categories, and print all weights, gradients, and updated weights for complete transparency into the training process!